

// SOMMAIRE

Table des matières

01	Introduction aux API	Définitions, rôle, formats d'échange	p.02
02	Architecture REST	Principes, verbes HTTP, codes de statut	p.03
03	Concevoir une API REST	Ressources, pagination, versioning, HATEOAS	p.04
04	GraphQL — fondamentaux	Schéma, types, queries, mutations	p.05
05	GraphQL — avancé	Resolvers, N+1, pagination, subscriptions	p.06
06	REST vs GraphQL & sécurité	Comparatif, OWASP API Top 10	p.07
07	Ressources & bonnes pratiques	Outils, documentation, pour aller plus loin	p.08

Qu'est-ce qu'une API ?

Une **API** (Application Programming Interface) est un contrat qui permet à deux systèmes logiciels de communiquer selon des règles définies : quelles données sont disponibles, comment les demander, sous quel format elles sont retournées. Une **API web** expose généralement ce contrat au-dessus du protocole **HTTP**, et échange des données le plus souvent au format **JSON**.

Pourquoi une API ?

- Découpler le frontend du backend (web, mobile, IoT)
- Permettre l'intégration avec des partenaires tiers
- Exposer des microservices de façon standardisée
- Réutiliser une même logique métier sur plusieurs clients

Deux grandes approches

Style	Principe
REST	Ressources exposées via des URL et verbes HTTP standards
GraphQL	Langage de requête unique permettant au client de choisir précisément les données voulues

Formats d'échange courants

Format	Usage typique
JSON	Standard de facto pour REST et GraphQL, léger et lisible
XML	API historiques, SOAP, certains systèmes d'entreprise
Protocol Buffers	Format binaire utilisé par gRPC, très performant

À retenir — REST et GraphQL ne sont pas des protocoles mais des **styles d'architecture** : ils s'appuient tous les deux sur HTTP, mais définissent des conventions très différentes pour structurer les échanges.

// MODULE 01

Architecture REST

REST (Representational State Transfer) est un style d'architecture défini par Roy Fielding en 2000. Une API est dite RESTful lorsqu'elle respecte un ensemble de contraintes qui la rendent simple, prévisible et évolutive.



Verbes HTTP & sémantique

Méthode	Action	Idempotente
GET	Lire une ressource	Oui
POST	Créer une nouvelle ressource	Non
PUT	Remplacer intégralement une ressource	Oui
PATCH	Modifier partiellement une ressource	Non (en général)
DELETE	Supprimer une ressource	Oui

Codes de statut essentiels

Code	Signification
200 / 201	Succès / Ressource créée
204	Succès, sans contenu retourné
400	Requête invalide
401 / 403	Non authentifié / Accès refusé
404	Ressource introuvable
500	Erreur serveur

```
# Exemple de requête REST
GET /api/v1/users/42 HTTP/1.1
Host: api.exemple.com
Authorization: Bearer <token>

# Réponse
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 42,
  "name": "Alice"
}
```

« Le niveau 3 du modèle de maturité de Richardson (HATEOAS) est rarement atteint en pratique, mais il reste l'idéal théorique d'une API pleinement RESTful. »

// MODULE 02

Concevoir une API REST propre

Nommage des ressources

Les URL doivent représenter des **ressources** (noms), jamais des actions (verbes) — l'action est déjà portée par la méthode HTTP.

```
# Bonnes pratiques
GET /users
GET /users/42/orders
POST /users
DELETE /users/42
```

```
# À éviter
GET /getUser?id=42
POST /createUser
GET /deleteUser/42
```

Pagination, filtrage & tri

Technique	Exemple
Pagination offset	GET /articles?page=2&limit=20
Pagination par curseur	GET /articles?after=eyJpZCI6NDJ9&limit=20
Filtrage	GET /articles?status=published&author=alice
Tri	GET /articles?sort=-created_at

Versioning

- **Dans l'URL** — /api/v1/... (le plus courant, simple)
- **Dans le header** — Accept: application/vnd.api+json;version=2
- **Query param** — ?version=2 (moins recommandé)

Authentification

- **API Key** — simple, adaptée aux intégrations serveur-serveur
- **OAuth 2.0 / JWT** — standard pour les API grand public
- **mTLS** — authentification mutuelle pour environnements sensibles

HATEOAS

Hypermedia As The Engine Of Application State : la réponse inclut les liens vers les actions possibles, rendant l'API auto-descriptive.

```
{
  "id": 42, "status": "pending",
  "_links": {
    "self": { "href": "/orders/42" },
    "cancel": { "href": "/orders/42/cancel" }
  }
}
```

Bonne pratique — Documentez systématiquement l'API avec la spécification **OpenAPI (Swagger)** : elle permet de générer automatiquement documentation interactive, clients SDK et tests de contrat.

// MODULE 03

GraphQL — les fondamentaux

GraphQL, créé par Facebook en 2012 puis open-sourcé en 2015, est un langage de requête pour API. Contrairement à REST, il expose **un seul point d'entrée** (généralement `/graphql`) et laisse le client préciser exactement les champs souhaités.

Le schéma : contrat typé

Toute API GraphQL repose sur un **schéma** fortement typé, écrit en SDL (Schema Definition Language), qui décrit les types disponibles et les opérations possibles.

```
# schema.graphql
type User {
  id: ID!
  name: String!
  orders: [Order!]!
}

type Query {
  user(id: ID!): User
  users(limit: Int = 20): [User!]!
}

type Mutation {
  createUser(name: String!): User!
}
```

Les 3 opérations

Opération	Rôle
Query	Lire des données
Mutation	Créer / modifier / supprimer
Subscription	Recevoir des mises à jour en temps réel

Requête côté client

```
query {
  user(id: "42") {
    name
    orders { id total }
  }
}
```

Sur-lecture et sous-lecture : le problème que GraphQL résout

● over-fetching (REST)

Un endpoint `/users/42` renvoie souvent **plus de champs** que nécessaire pour l'écran affiché, gaspillant bande passante et temps de traitement mobile.

● under-fetching (REST)

À l'inverse, afficher un profil avec ses commandes nécessite souvent **plusieurs appels** (`/users/42` puis `/users/42/orders`).

GraphQL — une seule requête, un seul aller-retour réseau, exactement les champs demandés : ni plus, ni moins.

// MODULE 04

GraphQL — concepts avancés

Resolvers : la logique derrière le schéma

Chaque champ du schéma est associé à une fonction appelée **resolver**, responsable d'aller chercher la donnée (base de données, autre API, cache...).

```
// resolvers.js
const resolvers = {
  Query: {
    user: (_, { id }, ctx) => ctx.db.users.findById(id),
  },
  User: {
    orders: (user, _, ctx) => ctx.db.orders.findByUser(user.id),
  }
};
```

Le problème N+1 & DataLoader

● le problème

Une requête listant 50 utilisateurs avec leurs commandes peut déclencher **1 requête pour la liste + 50 requêtes individuelles** pour les commandes — un appel base de données par résolution de champ imbriqué.

● la solution : batching

La librairie **DataLoader** regroupe (*batch*) et met en cache les requêtes similaires survenant dans le même cycle d'exécution, ramenant l'exemple à **2 requêtes** au total.

Pagination par curseur (Relay-style)

```
query {
  users(first: 10, after: "cursor_42") {
    edges { node { id name } cursor }
    pageInfo { hasNextPage endCursor }
  }
}
```

Sécurité spécifique à GraphQL

Risque	Mitigation
Requêtes trop profondes / complexes (DoS)	Limiter la profondeur et calculer un coût de requête
Introspection exposée en production	Désactiver l'introspection publique en prod
Batching abusif (aliasing)	Limiter le nombre d'alias par requête

// MODULE 05

REST vs GraphQL & sécurité des API

REST

- Multiples endpoints, cache HTTP natif
- Simple à mettre en cache (CDN, navigateur)
- Idéal pour API publiques simples et CRUD
- Sur/sous-lecture fréquente

VS

GraphQL

- Point d'entrée unique, requêtes précises
- Cache plus complexe (par champ, pas par URL)
- Idéal pour clients riches (mobile, agrégation)
- Complexité serveur accrue (resolvers, N+1)

Critère	REST	GraphQL
Courbe d'apprentissage	Faible	Moyenne
Nombre d'appels réseau	Souvent plusieurs	Un seul, généralement
Cache HTTP standard	Natif	Nécessite une couche dédiée
Upload de fichiers	Natif (multipart)	Nécessite une extension
Versioning	Explicite (v1, v2...)	Évolution continue du schéma

OWASP API Security Top 10 (extraits)

#	Risque
API1	Broken Object Level Authorization — absence de vérification que l'utilisateur a le droit d'accéder à l'objet demandé
API2	Broken Authentication — mécanismes d'authentification faibles ou mal implémentés
API3	Broken Object Property Level Authorization — exposition de champs sensibles (over-fetching non contrôlé)
API4	Unrestricted Resource Consumption — absence de rate limiting, requêtes coûteuses non bornées
API8	Security Misconfiguration — introspection GraphQL ouverte, CORS trop permissif, headers manquants

À retenir — Quel que soit le style choisi, l'authentification (OAuth2/JWT), le **rate limiting**, la validation stricte des entrées et le principe du moindre privilège restent les fondations de toute API sécurisée.

// MODULE 06

Ressources & bonnes pratiques

Outils incontournables

Outil	Usage
Postman / Insomnia	Test manuel et automatisé d'API REST et GraphQL
Swagger / OpenAPI	Spécification et documentation interactive REST
GraphiQL / Apollo Studio	Explorateur interactif de schéma GraphQL
curl / HTTPie	Requêtes en ligne de commande
Apollo Server / GraphQL Yoga	Serveurs GraphQL populaires côté Node.js

Frameworks REST

- Express / Fastify (Node.js)
- Django REST Framework (Python)
- Spring Boot (Java)
- Laravel (PHP)

Écosystème GraphQL

- Apollo Client (frontend)
- Relay (React, Facebook)
- Hasura / PostGraphile (auto-génération)
- GraphQL Federation (multi-services)

Pour approfondir

- Documentation officielle graphql.org
- Spécification OpenAPI (swagger.io)
- OWASP API Security Project
- REST API Design Rulebook

En résumé

01 REST = ressources + verbes HTTP + statuts standards

02 GraphQL = un endpoint + requêtes précises typées

03 Documenter systématiquement (OpenAPI / schéma)

04 Paginer, filtrer, versionner dès la conception

05 Surveiller le coût des requêtes GraphQL

06 Sécuriser : auth, rate limiting, moindre privilège

● conclusion

REST et GraphQL ne s'opposent pas : ce sont deux outils avec des cas d'usage différents, parfois combinés dans une même architecture (ex. GraphQL en façade d'agrégation devant des microservices REST). Le bon choix dépend des clients à servir, de la complexité des données et des contraintes de cache et de performance.

--- HTTP/1.1 200 OK · end_of_course ---

[HAPPY BUILDING]